

FINAL YEAR PROJECT · TECHNICAL REFERENCE

Understanding BullMQ

A complete guide to what BullMQ is, where and how it is used inside the **Google Meet Meeting Bot** backend, why it was chosen, and the benefits it provides.

Repository: Zainab-Junaid / Final-Year-Project
Generated: 2026-05-04

1 · What is BullMQ?

BullMQ is an open-source *Node.js job-queue library* backed by **Redis**. It was created as the next-generation successor to the popular Bull library and is maintained by the same team (Taskforce.sh).



npm package

`npm install bullmq` — TypeScript typings are included out of the box.

At its core, BullMQ solves a very common backend problem:

"I need to do a slow or risky operation (AI call, email, file conversion) but I cannot block the HTTP response waiting for it to finish."

BullMQ lets you **enqueue** a piece of work (a *Job*) into a persistent *Queue* stored in Redis, and then have one or more separate *Worker* processes pick up and execute those jobs in the background — with automatic retries, priority ordering, progress tracking, and concurrency control.

Core Concepts

Concept	What it is
Queue	A named list in Redis. Producers add jobs to it.
Job	A unit of work with a name, payload data, and metadata (attempts, progress, result).
Worker	A process that listens on a queue and runs your processor function for each job.
QueueEvents	Event emitter that broadcasts completed / failed / progress events across processes.

FlowProducer

Advanced: creates job trees (parent–child dependencies). Not used in this project.

2 · Where Is BullMQ Used in This Project?

BullMQ lives exclusively inside the `google-meet-meeting-bot/` backend service. It is **not used** in the Next.js frontends (`frontend_2/` , `frontend_3/`) or in the Chrome Extension.

Relevant Files

File	Role
<code>src/infrastructure/queue/meetingQueue.ts</code>	Producer — creates the queue, enqueues jobs, exposes helpers to check/retry job status and get queue stats.
<code>src/infrastructure/workers/meetingProcessingWorker.ts</code>	Worker / Consumer — runs the 3-stage post-meeting processing pipeline for each job.
<code>src/config/constants.ts</code>	Stores queue names (<code>QueueNames</code>) and retry configuration (<code>JobRetryConfig</code>).

Queue Name and Retry Config (from constants.ts)

```
// Queue names
export const QueueNames = {
  MEETING_PROCESSING: 'meeting-processing',
  EMAIL_NOTIFICATIONS: 'email-notifications', // reserved for
  future use
} as const;

// Retry settings
export const JobRetryConfig = {
```

```
MAX_ATTEMPTS: 3,  
BACKOFF_TYPE: 'exponential' as const,  
BACKOFF_DELAY_MS: 5000,           // 5s → 10s → 20s  
REMOVE_ON_COMPLETE: 100,         // keep last 100 completed jobs in  
Redis  
REMOVE_ON_FAIL: 500,             // keep last 500 failed jobs for  
debugging  
};
```

Why NOT Used for Caption Buffering

The caption data streaming from the bot (real-time captions) uses plain **Redis Lists** (`RPUSH`), not BullMQ. The code in `captionBuffer.ts` explicitly explains this design choice:

⚠ Design Decision — Caption Buffer

BullMQ would be *overkill* for simple write-buffering of caption segments. Redis Lists offer $O(1)$ atomic appends, no extra overhead, and the only need is "buffer until flush" — no retry logic or job scheduling required there.

3 · How Is BullMQ Used? (Step by Step)

Step 1 — Queue Initialisation (Producer)

The queue is created lazily (only once) inside `getMeetingProcessingQueue()`. It connects to the same Redis instance used by the caption buffer.

```
meetingQueue = new Queue<MeetingProcessingJobData,
ProcessingJobResult>(
  QueueNames.MEETING_PROCESSING,
  {
    connection: redis.options,
    defaultJobOptions: {
      attempts: JobRetryConfig.MAX_ATTEMPTS,          // 3
      tries
      backoff: {
        type: JobRetryConfig.BACKOFF_TYPE,           //
        'exponential'
        delay: JobRetryConfig.BACKOFF_DELAY_MS,      // 5000
        ms
      },
      removeOnComplete: { count: 100 },
      removeOnFail:    { count: 500 },
    },
  },
);
```

Step 2 — Enqueuing a Job (Trigger)

When the bot finishes recording a meeting (via `POST /bot-done`), the backend calls:

```
await enqueueMeetingProcessing(meetingId, 'normal');
```

This adds a job named `"process-completed-meeting"` with:

- **Payload:** `{ meetingId, timestamp, priority }`
- **Unique Job ID:** `"meeting-{meetingId}"` — prevents duplicates
- **Priority:** mapped to BullMQ numbers (high=1, normal=2, low=3)

✓ Deduplication

If `/bot-done` is accidentally called twice for the same meeting, BullMQ throws `JobIdAlreadyExistsError` and the second call is silently ignored — no duplicate processing.

Step 3 — Worker Starts

A separate Node.js process (or thread) calls `startMeetingProcessingWorker()`, which registers a BullMQ `worker` on the same queue name:

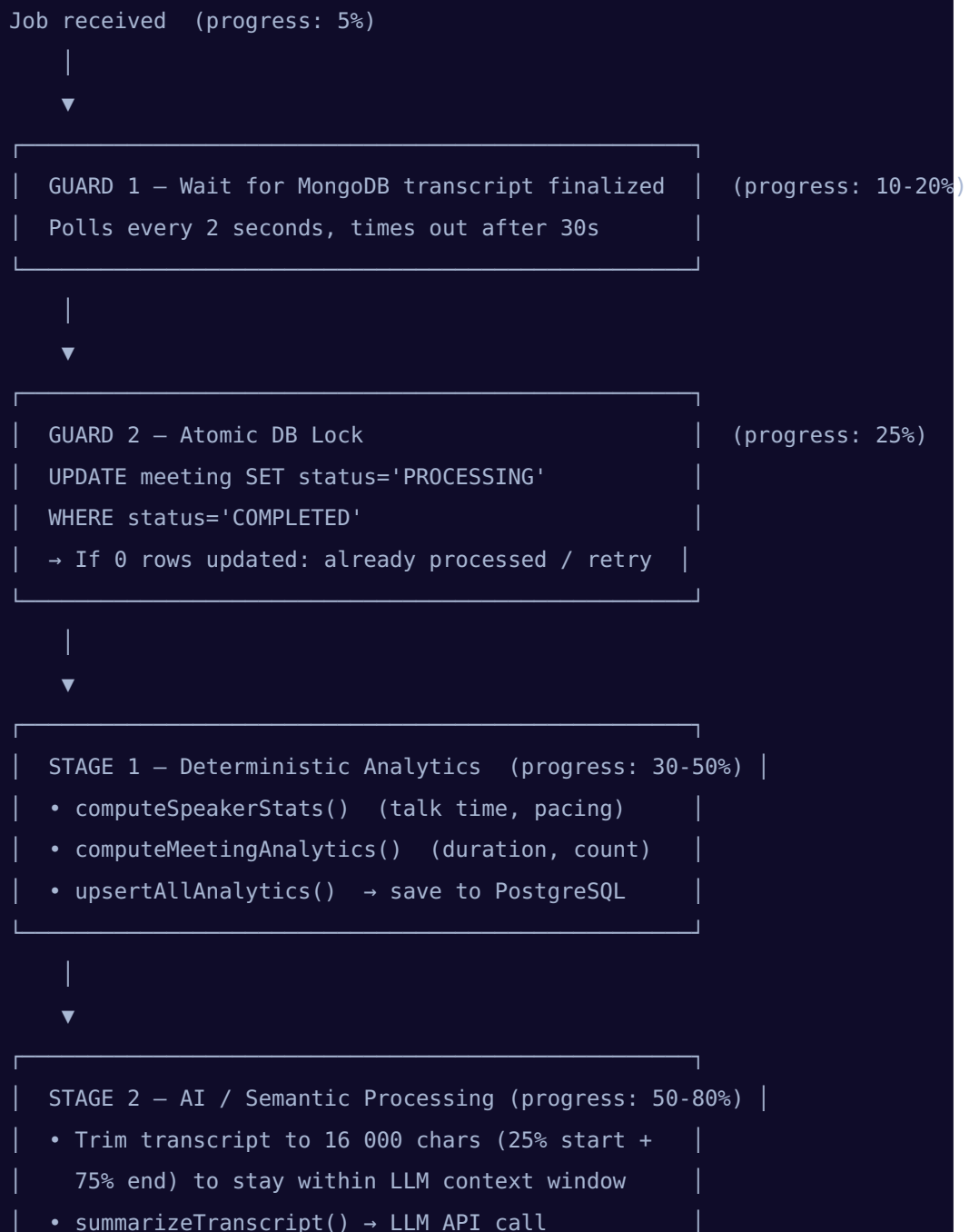
```
worker = new Worker<...>(
  QueueNames.MEETING_PROCESSING,
  processMeetingJob,      // your async processor function
  {
    connection: redis.options,
    concurrency: 2,      // process up to 2 meetings in parallel
  }
);
```

Event listeners are also registered:

```
worker.on('completed', (job, result) => { ... });
worker.on('failed',    (job, error)  => { ... });
worker.on('stalled',   (jobId)       => { ... });
worker.on('error',     (error)       => { ... });
```

4 · The 3-Stage Processing Pipeline

The worker function `processMeetingJob(job)` runs the following pipeline. Progress is reported to BullMQ at each stage so it can be surfaced to monitoring dashboards.




```
| • saveSummary() + saveActionItems() → PostgreSQL |
| • upsertMeetingKeywords() → PostgreSQL           |
| (failure here is non-fatal – meeting still saved) |
|_____|
|
| ▼
|_____|
| STAGE 3 – Finalization          (progress: 100%) |
| UPDATE meeting SET status='PROCESSED'           |
|           segmentCount = transcript.segments.length |
|_____|
|
| ▼
Job COMPLETED ✅ – result object returned to BullMQ
```

⚠ Rollback on Failure

If any unhandled error is thrown in Stages 1–3, the worker resets the meeting status back to `COMPLETED` and re-throws the error. BullMQ then marks the job as **FAILED** and schedules a retry with exponential backoff.

5 · Full System Architecture





Socket.IO event → Dashboard

6 · Why Was BullMQ Chosen?

Alternatives Considered

Option	Why Rejected
Process directly in <code>bot-done</code> handler	AI API calls take 30–60+ seconds. HTTP response would time out. Bot container would be blocked.
Redis Pub/Sub	Fire-and-forget. If the worker is temporarily down when the event fires, the message is permanently lost . No retry capability.
Plain Redis Lists (RPUSH/LPOP)	No retry logic, no job status, no concurrency control, no deduplication. Great for caption buffering but too primitive for the processing pipeline.
PostgreSQL-backed queue (e.g. pg-boss)	Would work, but adds load to the primary database. Redis is already in the stack, so BullMQ has zero extra infrastructure cost.
BullMQ 	Persistent jobs in Redis, automatic retries, deduplication, concurrency, progress tracking — all with a clean TypeScript API.

The Core Reason

Post-meeting processing is **slow** (AI calls), **risky** (network failures), and **exactly-once** (must not duplicate summaries or analytics). BullMQ's combination of persistence, deduplication via `jobId`, and atomic retries addresses all three constraints simultaneously.

7 · Benefits of Using BullMQ

1 · Automatic Retries with Exponential Backoff

Jobs that fail due to transient errors (API down, DB connection blip) are automatically retried up to 3 times with delays of 5 s → 10 s → 20 s. No manual polling or cron job needed.

2 · Job Deduplication

Every meeting is identified by a unique job ID (`meeting-{meetingId}`). Even if `/bot-done` is called multiple times (network retry, bug), BullMQ silently rejects the duplicate. The user never sees a duplicate summary.

3 · Non-blocking HTTP Layer

The Express endpoint returns `200 OK` in <1 ms. All heavy lifting happens asynchronously in the worker. The bot container is freed immediately to leave the meeting room.

4 · Job Status Visibility

At any time, the system can query the state of any job:

```
const status = await getMeetingJobStatus(meetingId);  
// { exists: true, state: 'active', progress: 50,  
//   attemptsMade: 1, failedReason: undefined }
```

States available: `waiting` → `active` → `completed` / `failed` / `delayed`

5 · Real-time Progress Tracking

`job.updateProgress(n)` is called at each stage (5 → 10 → 25 → 50 → 80 → 100). This can be exposed to the frontend as a live progress bar.

6 · Concurrency Control

`concurrency: 2` means exactly 2 meetings are processed in parallel. Raising this number (with no code change) instantly scales throughput. Lowering it protects downstream APIs from being overwhelmed.

7 · Horizontal Scalability

Because all state lives in Redis (not in process memory), you can run the worker on multiple machines. BullMQ uses atomic Redis commands to ensure only one worker claims each job — no coordination code needed.

8 · Auto-cleanup of Old Jobs

`removeOnComplete: 100` keeps only the last 100 completed jobs in Redis.
`removeOnFail: 500` keeps the last 500 failures for post-mortem debugging. Redis memory usage stays bounded without manual cleanup.

9 · Graceful Shutdown

`stopMeetingProcessingWorker()` calls `worker.close()`, which waits for active jobs to finish before stopping. Deployed on `SIGTERM` (Kubernetes/Docker), this prevents in-flight jobs from being orphaned.

10 · Great TypeScript Support

BullMQ ships its own TypeScript generics: `Queue<TData, TResult>`, `Worker<TData, TResult>`, `Job<TData, TResult>` — giving full type safety on job payloads and results with zero extra configuration.

8 · Quick Reference Cheat Sheet

Add a job to the queue

```
const job = await queue.add('job-name', { meetingId }, { jobId: 'unique-id' });
```

Start a worker

```
const worker = new Worker('queue-name', async (job) => {  
  // do work  
  await job.updateProgress(50);  
  return { success: true };  
}, { connection: redis, concurrency: 2 });
```

Check job state

```
const job = await queue.getJob('unique-id');  
const state = await job.getState();  
// 'waiting' | 'active' | 'completed' | 'failed' | 'delayed'
```

Retry a failed job

```
await job.retry();
```

Get queue statistics

```
const counts = await
queue.getJobCounts('waiting','active','completed','failed');
// { waiting: 0, active: 2, completed: 47, failed: 1 }
```

Graceful shutdown

```
await worker.close(); // waits for active jobs to finish
await queue.close();  // closes the Redis connection
```

Job Lifecycle (State Machine)

